

SWE642

EXTRA CREDIT ASSIGNMENT DOCUMENTATION

Group Members:

Swapneel Suhas Vaidya - G01459609
Greeshma Koneru - G01434318
Jyoshmitha Reddy Paturi - G01459587
Charitha Appagari - G01457269
Bhuvana Venigalla - G01448139
Chandana Konda - G01449289
K.V.Lokeswar Reddy - G01414088

Assignment Overview:

The assignment aims to provide broad practice in full-stack application development. In Homework 3, the front-end application will be implemented using the Angular framework, version 2 or higher, whereas the back-end functionality is supposed to be implemented as a set of RESTful web services. The back-end is based on the Spring Boot platform and uses JPA/Hibernate for persistence layer development.

Building on HW3, Extra Credit Assignment expands the project by adopting Vue.js as another front-end framework. (Reference: <https://vuejs.org/guide/>), re-implementing the Angular-based front-end in HW3 using Vue.js, and integrating the existing HW3 back-end application with the newly developed Vue.js application.

Technologies Used:

Frontend Technologies:

Angular - Used for the development of the original frontend for HW3.

Vue.js - re-implemented the front end in this assignment.

Vue Router - For client-side routing management.

Bootstrap - Making it look responsive and nice Bootstrap.

HTML5 & CSS3 - Core Web technologies for the structure and styling of the application.

JavaScript (ESNext) - To script both Angular and Vue.js, respectively.

Node.js - Used for the dependency manager and building Vue.js Frontend Application.

Backend Technologies:

Spring Boot - For implementing RESTful web services, ensuring a scalable and efficient backend.

Java - The programming language used in the backend.

Database:

MySQL - applied for data storage, retrieval, and management.

Development Tools & Build Tools:

npm (Node Package Manager) - for managing frontend dependencies and running build scripts.

Vue CLI - for scaffolding and managing applications written in Vue.js.

ESLint - For linting, and maintaining coding standards in Vue.js.

Babel - Used for transpiling JavaScript to browser-compatible.

Code Explanation and Flow

The application has been implemented with Vue.js on the frontend and Spring Boot as the backend, and integrated with RESTful APIs. Following is an explanation of the code flow and explanation of its functionality:

1. Frontend Code (Vue.js)

Component Structure:

- This application has a component-based architecture.
- It is done such that every feature of the application has been implemented in one.vue file.
- Reusable structures include survey form, list page, and home page.
- Components like SurveyPage.vue handle user input, data validation, and form submission.

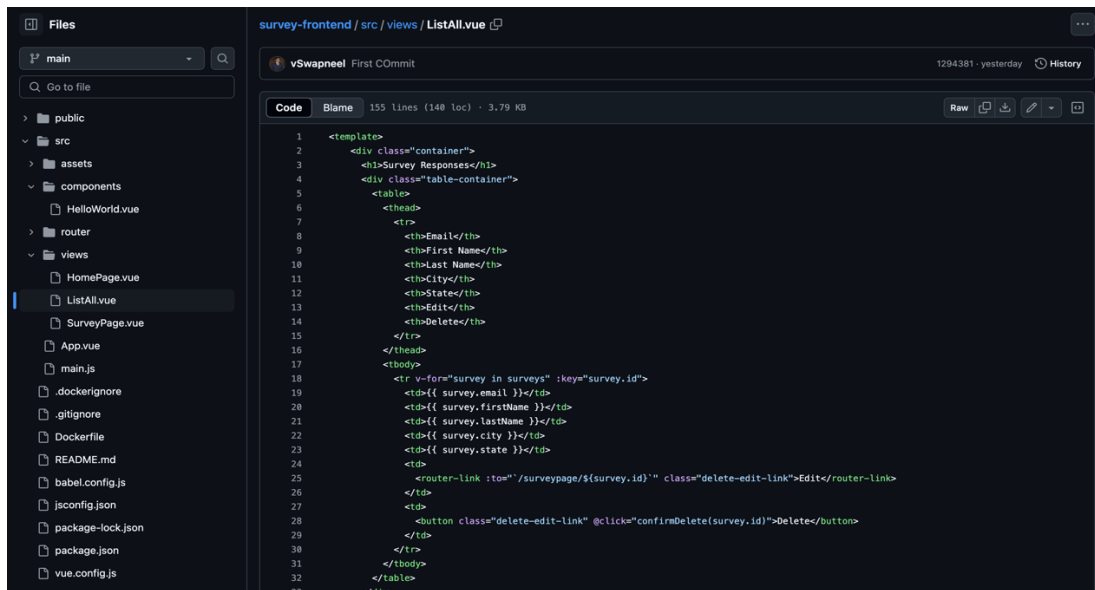
HomePage: This is the main page of our application that consists of navigation bar that includes SurveyPage and ListAll .

```
1 <template>
2 <div class="container mt-5">
3   <!-- Swapneel Suhas Vaidya -->
4   <!-- Home Page, Entry point for the application -->
5   <div class="text-center">
6     <h1>Welcome to the CS Department</h1>
7     <p class="lead">
8       Take our department
9       <router-link to="/surveypage">Survey</router-link>
10      and give us your feedback! You can check all the feedbacks on this
11      <router-link to="/listall">List</router-link>.
12    </p>
13  </div>
14 </div>
15 </template>
16
17 <script>
18 export default {
19   name: "HomePage", // Ensure the component name follows multi-word naming convention
20 };
21 </script>
22
23 <style scoped>
24 .container {
25   background: rgba(255, 255, 255, 0.6);
26   padding: 20px;
27   border-radius: 10px;
28   box-shadow: 0 2px 10px rgba(0, 0, 0, 0.1);
29   -webkit-backdrop-filter: blur(10px);
30   backdrop-filter: blur(10px);
31   position: absolute;
32   top: 30px;
33 }
```

SurveyPage: This page contains a form for users to submit survey data, with fields such as first name, last name, and others. It includes form validation and error handling.

```
1 <template>
2 <div class="container mt-navbar">
3   <!-- Swapneel Suhas Vaidya -->
4   <!-- Contains the survey element -->
5   <div class="header-box text-center">
6     <h1>CS Department Survey</h1>
7   </div>
8
9   <div id="form-errors" class="error mt-3">{{ zipError }}</div>
10
11   <form class="mt-4" @submit.prevent="onSubmit">
12     <div class="mb-3 element">
13       <label form="first-name" class="form-label">First Name:</label>
14       <input
15         type="text"
16         id="first-name"
17         class="form-control"
18         v-model="firstName"
19         name="firstName"
20         required
21         placeholder="Enter first name"
22         :disabled="!surveyId"
23       />
24       <div v-if="formErrors.firstName" class="text-danger">
25         First name is required.
26       </div>
27     </div>
28
29     <div class="mb-3 element">
30       <label form="last-name" class="form-label">Last Name:</label>
31       <input
32         type="text"
33         id="last-name"
34       />
35     </div>
36   </form>
37 </div>
```

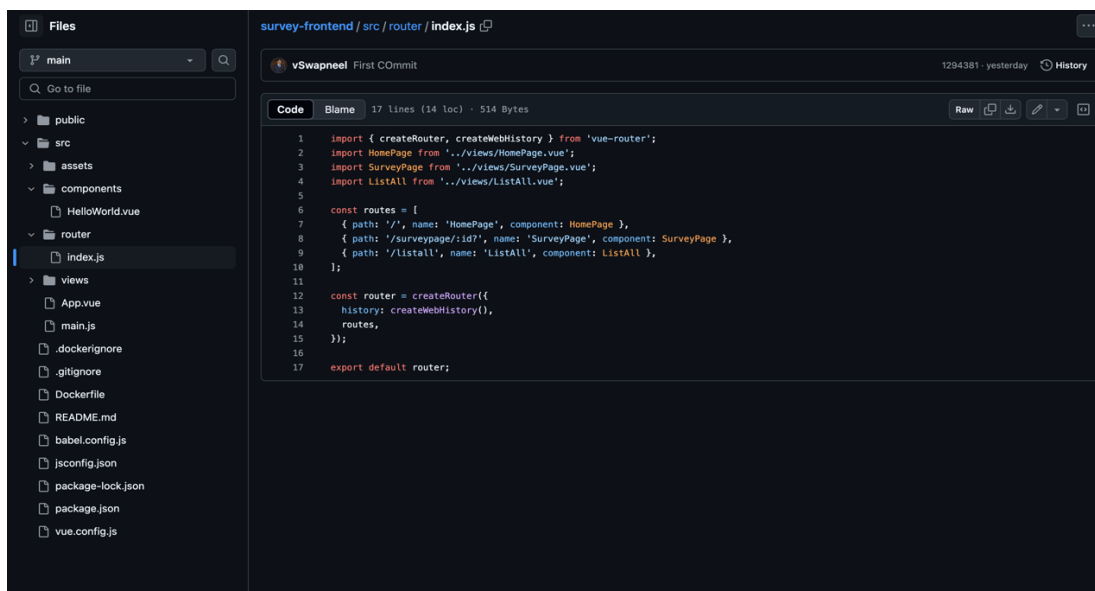
ListAll: It displays all submitted survey responses in a tabular format, with options to edit or delete individual responses.



```
1 <template>
2   <div class="container">
3     <h1>Survey Responses</h1>
4     <div class="table-container">
5       <table>
6         <thead>
7           <tr>
8             <th>Email</th>
9             <th>First Name</th>
10            <th>Last Name</th>
11            <th>City</th>
12            <th>State</th>
13            <th>Edit</th>
14            <th>Delete</th>
15          </tr>
16        </thead>
17        <tbody>
18          <tr v-for="survey in surveys" :key="survey.id">
19            <td>{{ survey.email }}</td>
20            <td>{{ survey.firstName }}</td>
21            <td>{{ survey.lastName }}</td>
22            <td>{{ survey.city }}</td>
23            <td>{{ survey.state }}</td>
24            <td>
25              <router-link :to="/surveypage/${survey.id}" class="delete-edit-link">Edit</router-link>
26            </td>
27            <td>
28              <button class="delete-edit-link" @click="confirmDelete(survey.id)">Delete</button>
29            </td>
30          </tr>
31        </tbody>
32      </table>
33    </div>
34  </div>
35</template>
```

Routing:

- Vue Router is used for client-side routing. Routes such as /surveypage/:id? and /listall manage application navigation without full page reloads.
- Routing configuration can be found in router/index.js.



```
1 import { createRouter, createWebHistory } from 'vue-router';
2 import HomePage from '../views/HomePage.vue';
3 import SurveyPage from '../views/SurveyPage.vue';
4 import ListAll from '../views/ListAll.vue';
5
6 const routes = [
7   { path: '/', name: 'HomePage', component: HomePage },
8   { path: '/surveypage/:id?', name: 'SurveyPage', component: SurveyPage },
9   { path: '/listall', name: 'ListAll', component: ListAll },
10 ];
11
12 const router = createRouter({
13   history: createWebHistory(),
14   routes,
15 });
16
17 export default router;
```

Directives:

The following Vue.js directives add interactivity to the components:

- v-model binds form fields to Vue's reactive data properties.
- v-if and v-else conditionally render elements.
- v-bind dynamically binds attributes or expressions to elements.

Data Flow:

- User inputs in the survey form are bounded to data properties by the v-model.
- Data validation and form errors are handled locally using computed properties and event listeners.

API Integration:

- The fetchSurveyData method fetches data from the Spring Boot back-end using the fetch API.
- onSubmit sends form data to the backend via POST or PUT, depending on if there is a survey id.

2. Backend Code (Spring Boot)**RESTful APIs:**

- Spring Boot acts as the backend framework that exposes endpoints to handle CRUD operations for surveys.
- APIs like /api/surveys for creating surveys and /api/surveys/{id} to fetch or update a survey are implemented.

Data Management:

- JPA/Hibernate handles persistence management; it will interact perfectly with the MySQL database.

3. Integrating Frontend and Backend

- The frontend uses the Fetch API to make HTTP requests to the backend.
- Data fetched from APIs are stored in Vue.js data properties and dynamically bound to UI elements using Vue's reactivity.
- For instance, the fetchSurveyData method populates the form fields with survey details for editing an existing survey.

4. Docker Configuration

The application is containerized using Docker:

- First stage will build the Vue.js frontend using the Node.js image.
- The second stage uses an Nginx image to serve the built application.
- This ensures a lightweight and production-ready deployment setup.

5. Summary of the Pages on the Frontend:

Home Page: Navigation's welcome screen or lander.

Survey Page: There is a form either for creating or editing a survey entry. Upon users' input, does form validation and API action upon submission of data to record.

List Page: Present listing of all surveys in one page, probably from an API request to the backend.

Final Output:

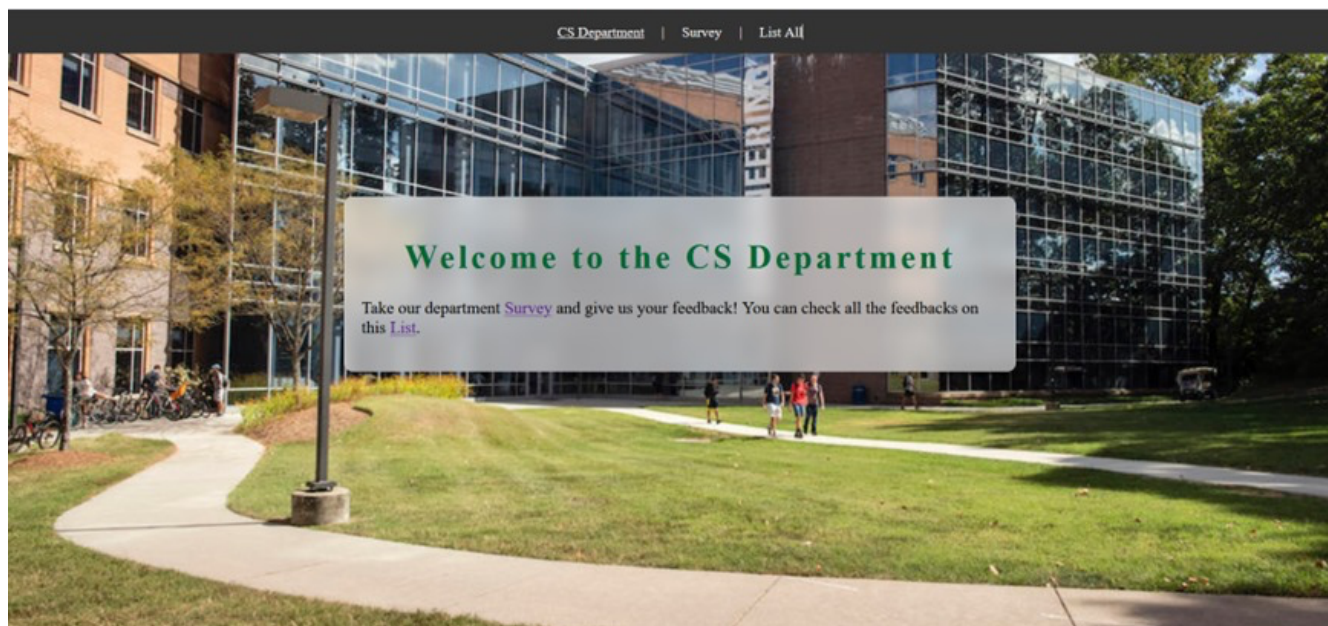
1. Welcome Page

Displayed Content:

- A hello message to welcome the user to the survey and view feedback.

Development:

- It uses Vue Router for routing and links to other pages, "Survey" and "List All".
- The background image is designed using CSS.
- The welcome text is rendered dynamically by using Vue.js and is stylized by using Bootstrap classes.



2. Survey Form Page

Displayed Content:

- A form that collects user inputs such as name, email, address, and survey preferences.
- Dynamic error messages for invalid fields.

Development:

- The form fields are bound to Vue's data properties through v-model.
- Validation is handled in methods with error states displayed using v-if directives.

API calls via the fetch function:

- POST for new survey submissions.

- PUT for updating existing entries based on a survey ID passed via the route parameters.

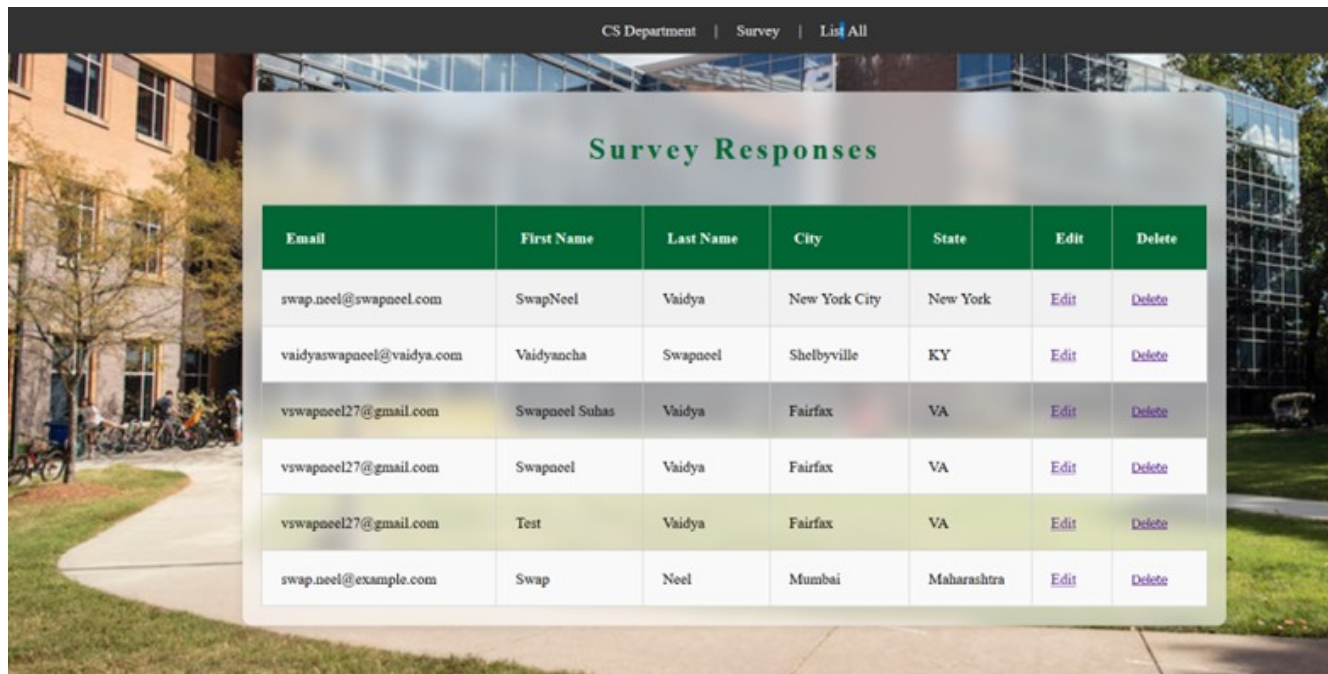
3. Survey Responses Page

Displayed Content:

- A tabular list of all the survey responses.
- Edit and Delete options to each record.

Development:

- The table is populated by pulling data from the backend.
- Edit links redirect to the Survey form pre-populated with existing data using Vue Router.
- Delete functionality and remove the entry dynamically from the UI.



4. API Calls (Postman Snippets)

➤ **GET Data:**

- Retrieves all the survey data from the backend.
- This integrates with the response into Vue.js's reactive data properties.

➤ **POST Data:**

- A new survey entry is forwarded to the backend.
- Form submission with POST containing data serialized to JSON.

➤ **PUT Data:**

- Makes updates on existing survey entries.
- Data is to be sent by putting in the `/api/surveys/{id}`.
- Data will come from the input values in a form that will be checked by me.

➤ **DELETE Data:**

- Deletion of specific surveys by a DELETE request.
- Removes the corresponding entry from the table without reloading the page.

survey-form / Get data

GET

http://localhost:8081/api/survey

Send

Params

Authorization

Headers (7)

Body

Scripts

Settings

Cookies

Query Params

Key	Value	Description	⌵ Bulk Edit
Key	Value	Description	

Body

Cookies

Headers (8)

Test Results (1/1)

200 OK · 116 ms · 2.48 KB

⌵ ⌵ ⌵

Pretty

Raw

Preview

Visualize

JSON

⌵

```
1 {
2
3   "id": 1,
4   "firstName": "Swapnil",
5   "lastName": "Vaidya",
6   "streetAddress": "123 Main St",
7   "city": "New York City",
8   "state": "New York",
9   "zip": "12345",
10  "telephone": "7754878473",
11  "email": "swapnil@example.com",
12  "surveyDate": "2023-10-01",
13  "lastVisit": "Department",
14  "interestSource": "Internet",
15  "recommendation": "Likely",
16  "comments": "Great course structure!"
17 }
```

⌵ Postbox

⌵ Run

⌵ Start Proxy

⌵ Cookies

⌵ Vault

⌵ Trash

survey-form / Post data

POST

http://localhost:8081/api/surveys

Send

Params

Authorization

Headers (8)

Body

Scripts

Settings

Cookies

☐ none

☐ form-data

☐ x-www-form-urlencoded

☒ raw

☐ binary

☐ GraphQL

JSON

Beautify

Body

Cookies

Headers (8)

Test Results (1/1)

200 OK · 123 ms · 593 B

⌵ ⌵ ⌵

Pretty

Raw

Preview

Visualize

JSON

⌵

```
1 {
2
3   "id": 22,
4   "firstName": "Swap",
5   "lastName": "Nani",
6   "streetAddress": "123 Main St",
7   "city": "Mumbai",
8   "state": "Maharashtra",
9   "zip": "400001",
10  "telephone": "123-456-7890",
11  "email": "swap.nani@example.com",
12  "surveyDate": "2023-10-07T00:00:00.000+00:00",
13  "lastVisit": "Campus",
14  "interestSource": "Internet",
15  "recommendation": "Very Likely",
16  "comments": "Great course!"
17 }
```

survey-form / **Delete data** Save Share

DELETE

http://localhost:8081/api/surveys/23

Send

Params Authorization Headers (7) Body Scripts Settings Cookies

Query Params

Key	Value	Description	Bulk Edit
Key	Value	Description	

Body Cookies Headers (6) Test Results (1/1) 204 No Content · 16 ms · 201 B Expand Refresh Search

Pretty Raw Preview Visualize Text Copy

1

survey-form / **Update data** Save Share

PUT

http://localhost:8081/api/surveys/13

Send

Params Authorization Headers (6) Body Scripts Settings Cookies

☐ none ☐ form-data ☒ www-form-urlencoded ☐ raw ☐ binary ☐ GraphQL ☐ JSON Beautify

1 {
2 "id": 13,
3 "firstName": "New Seagreen1",
4 "lastName": "valdya",
5 "streetAddress": "9443 Lee Highway",
6 "city": "Fallside",
7 "state": "VA",
8 "zip": "22081",
9
10 }
11
12 }
13
14
15

Body Cookies Headers (6) Test Results (1/1) 200 OK · 12 ms · 597 B Expand Refresh Search

Pretty Raw Preview Visualize JSON Copy

1 {
2 "id": 13,
3 "firstName": "New Seagreen1",
4 "lastName": "valdya",
5 "streetAddress": "9443 Lee Highway",
6 "city": "Fallside",
7 "state": "VA",
8 "zip": "22081",
9 "telephone": "334067898",
10 "email": "seagreen1@seagreen1.com",
11 "surveyDate": "2024-11-10T00:00:00-00:00",
12 "likeMost": "Campus",
13 "interestSource": "Internet",
14 "recommendation": "pretty like it",
15 "comments": "hahats up!!!"
16}

Position Runner Start Proxy Cookies View Test